Tailwind CSS Tutorial: Intermediate Level

For people comfortable with core utility classes, Flexbox/Grid utilities, states, and responsive prefixes, who want more control and less repetition.

Table of Contents

---

1. Customizing the Theme

`tailwind.config.js` is where a project's design system lives. Rather than reaching for arbitrary values everywhere, extend the theme with your own tokens:

```
// tailwind.config.js
module.exports = {
  theme: {
    extend: {
      colors: {
        brand: {
          light: '#7dd3fc',
          DEFAULT: '#0ea5e9',
          dark: '#0369a1',
        },
      },
      spacing: {
        '18': '4.5rem',
      },
      fontFamily: {
        display: ['Poppins', 'sans-serif'],
      },
    },
  },
};
```

Once extended, these become real utility classes:

```
<div class="bg-brand text-white p-18 font-
display">
```

`extend` adds to Tailwind's defaults rather than replacing them, so you keep the full default scale plus your additions.

2. Arbitrary Values

Sometimes a design calls for a value that isn't on the default scale. Square brackets let you drop in any exact value without touching the config:

```
<div class="w-[437px] top-[13px] bg-[#1da1f2]
text-[22px]">
```

Useful, but treat it as an escape hatch rather than a default habit — reaching for arbitrary values constantly usually means the project's theme scale needs extending instead (see Section 1), so the same values stay consistent and reusable.

---

3. Extracting Repeated Patterns with `@apply`

When the same cluster of utilities shows up across many elements, `@apply` lets you fold them into a single custom class inside your CSS:

```
/* input.css */
.btn-primary {
  @apply bg-blue-600 hover:bg-blue-700 text-
white font-semibold py-2 px-4 rounded-lg
transition;
}
```

```
<button class="btn-primary">Save</button>
```

This is best reserved for things that repeat identically across the whole app (buttons, badges, form inputs) — for anything that varies even slightly between uses, plain utility classes in the HTML are usually easier to keep track of than a growing pile of `@apply` classes.

---

4. Dark Mode

Tailwind ships dark mode support out of the box, toggled either by the user's system preference or manually:

```
// tailwind.config.js
module.exports = {
  darkMode: 'class', // or 'media' to follow
system preference automatically
};
```

With `class` mode, add/remove a `dark` class on `<html>` (commonly via a small bit of JavaScript tied to a toggle button), then style both states directly in your HTML:

```
<div class="bg-white text-gray-900 dark:bg-gray-
900 dark:text-gray-100">
  Adapts automatically based on the dark class
</div>
```

`class` mode is generally preferred over `media` in real projects since it lets users override the system setting with

an in-app toggle.

---

5. `group` and `peer`

These two variants let an element's style respond to a **relative** rather than to itself — a common need that plain CSS states can't do without extra classes.

`group` — style a child based on the state of a parent:

```
<div class="group border rounded-lg p-4
hover:bg-gray-50">
  <h3 class="text-gray-900 group-hover:text-
blue-600">Title</h3>
  <p class="text-gray-500 group-hover:text-gray-
700">Description</p>
</div>
```

Hovering anywhere on the card changes the title and description color, even though the `hover:` state was triggered on the parent.

`peer` — style an element based on a sibling's state, common for form validation UI:

```
<input type="email" class="peer border rounded
p-2" required>
<p class="hidden peer-invalid:block text-red-500
text-sm">
  Please enter a valid email
</p>
```

---

6. Transitions and Animations

```
<button class="transition duration-300 ease-in-
out hover:scale-105 hover:shadow-xl">
  Hover Me
</button>

<div class="animate-spin h-6 w-6 border-4
border-blue-500 rounded-full"></div>
<div class="animate-pulse bg-gray-200 h-4 w-full
rounded"></div>
```

`transition` alone enables smooth changes for common properties; add `duration-*`, `ease-*`, and `delay-*` to fine-tune it. Built-in `animate-*` utilities (`spin`, `ping`, `pulse`, `bounce`) cover most common loading/attention effects without writing a single `@keyframes` block yourself.

---

7. Gradients, Rings, and Shadows

```
<div class="bg-gradient-to-r from-purple-500
via-pink-500 to-red-500 text-white p-6">
  Gradient background
</div>

<button class="ring-2 ring-blue-500 ring-offset-
2 rounded-md p-2">
  Focus ring styling
</button>

<div class="shadow-md hover:shadow-2xl
transition-shadow">
```

```
  Elevates on hover
</div>
```

`ring-*` is worth knowing specifically for accessible focus states — it draws an outline-like ring without disturbing layout the way a `border` change would.

---

8. Aspect Ratio and Object Fit

Common for media-heavy layouts (thumbnails, video embeds):

```
<div class="aspect-video bg-gray-200">
  <!-- maintains a 16:9 box regardless of
content -->
</div>

<img src="photo.jpg" class="aspect-square
object-cover rounded-lg">
```

`object-cover` crops the image to fill its box without distorting it — the utility-class equivalent of `object-fit: cover;`.

---

9. The Official Plugins

Tailwind's core stays deliberately small; official plugins add functionality for common needs:

```
npm install -D @tailwindcss/typography
@tailwindcss/forms
```

```
// tailwind.config.js
module.exports = {
  plugins: [
    require('@tailwindcss/typography'),
    require('@tailwindcss/forms'),
  ],
};
```

- `@tailwindcss/typography` — adds a single `prose` class that nicely styles raw HTML content (blog posts, CMS output) without hand-styling every tag:

  ```
  <article class="prose lg:prose-xl">
    <!-- headings, paragraphs, lists all get
  sensible default styling -->
  </article>
  ```

- `@tailwindcss/forms` — normalizes form elements across browsers so `<input>`, `<select>`, and `<textarea>` look consistent as a starting point.

---

10. Custom Variants

Beyond `hover:` and `focus:`, you can define your own reusable variants for repeated conditional patterns:

```
// tailwind.config.js
const plugin = require('tailwindcss/plugin');
```

```
module.exports = {
  plugins: [
    plugin(function ({ addVariant }) {
      addVariant('optional', '&:optional');
      addVariant('third-child', '&:nth-
child(3)');
    }),
  ],
};
```

```
<input class="border optional:border-dashed">
```

---

11. Organizing Class Lists in Practice

Long class strings are the most common intermediate-level complaint. A few habits that help:

- **Order consistently** — layout → spacing → sizing → typography → color → state. Most editors' Tailwind plugins (like Prettier's Tailwind plugin) will auto-sort this for you.
- **Extract components, not just classes** — in a component-based framework (React, Vue), a repeated card pattern usually belongs in a `<Card>` component, not a repeated class string copy-pasted across files.
- **Reserve `@apply` for truly global, unchanging patterns**, and prefer components for anything with props/variants.

---

12. Practice Project

Build a themeable pricing table:

1. Extend `theme.colors` in the config with a custom `brand` color, and use it throughout.
2. Build three pricing cards using Flexbox/Grid utilities, with the "featured" card using `ring-2 ring-brand` and a slightly larger `scale-105`.
3. Add `dark:` variants throughout, toggled by adding/removing a `dark` class on `<html>`.
4. Use `group` so hovering a card also changes its "Choose Plan" button color via `group-hover:`.
5. Extract the repeated button styles into a `.btn` class with `@apply`.

---

13. Where to Go Next

- **Headless UI or Radix + Tailwind** — accessible unstyled component primitives (dropdowns, modals, tabs) styled entirely with utilities
- **The Tailwind Prettier plugin** for automatic class sorting
- **Just-in-Time engine internals** — understanding how Tailwind generates only the CSS you use helps explain why certain dynamic class names (built by string concatenation in JS) sometimes fail to appear
- **Design tokens shared with Figma** — some teams sync `tailwind.config.js` values directly from design tooling to keep design and code in lockstep

Tip: the jump from beginner to intermediate Tailwind is mostly about restraint — knowing when to extend the config vs. use an arbitrary value, and when to extract a component vs. keep inlining utilities.